

A PyQt5 Point-and-Click Front End for writing-habit and writing-schedule

Blaine Mooers

August 29, 2026

1 Summary

The two packages already separate their work cleanly into a parsing layer, a generation layer, and a thin `argparse` front end, so a graphical interface adds a third front end rather than a rewrite. The plan below places that interface in the `writing-habit` repository at `src/writing_habit/gui/`, behind an optional `[gui]` extra, and gives it two halves. The first half is a spreadsheet-style editor for the weekly template table, which is the single plain-text artifact both packages read. The second half is a command panel whose forms are generated by reading the existing `argparse` parsers, so every subcommand and every option becomes a widget without a hand-maintained duplicate list.

One design decision carries most of the value. The command panel builds itself from `writing_habit.cli.build_parser()` and `writing_schedule.cli.build_parser()` rather than from a hard-coded catalog of commands, because a hand-written catalog drifts the first time a flag is added. A second decision protects the command line as the reference interface. Every action in the graphical interface displays the exact shell command it is about to run, so the interface teaches the command line rather than hiding it.

The path you named, `~/6112MooersLabGitHubLabRepos/writing-schedule-py/writing_schedule`, holds the scheduler that owns the template table. The tracker that you named in the request lives beside it at `~/6112MooersLabGitHubLabRepos/writing-habit`. The plan covers both, because the template editor needs the scheduler parser and the command panel needs the tracker commands.

2 Decisions taken

The ten open questions of the first draft are now settled, and the plan below reflects the answers rather than the alternatives.

Every step of the build order below is complete. The interface edits the weekly table, runs every subcommand of both programs, shows what each run wrote, and is documented on the Sphinx site. Step 0 is the library work these decisions imply, step 1 is the window shell with its packaging, step 2 is the generated command panel that runs every tracker subcommand, and step 3 extends it to the scheduler. Every subcommand of both programs now runs by point and click, which leaves the weekly-table editor of steps 4 through 6.

3 What exists today

3.1 The two command surfaces

Every widget in the plan maps onto one of the subcommands below, which were read from the two `cli.py` files rather than from the documentation.

Two observations came out of that reading. The `dashboard` subcommand described in the project status note was absent from `writing_habit/cli.py`, although `dashboard.write_dashboard(con, week, out_path)` existed and the golden fixture test covered it. Step 0 added `writing-habit dashboard --week WEEK --out PATH --db DB`, so the table above lists it and the two front ends offer the same set of actions. The `writing-schedule` package

Table 1: The decisions that fix the design, and the consequence of each.

Decision	Consequence
PyQt5 is available to the interpreter you will use	no version negotiation is needed, and the pin stays at 5.15
The graphical layer ships inside <code>writing-habit</code>	one repository, one release, and an optional <code>[gui]</code> extra
PyQt5 under the GPL	the README states the split license, because the library and the command line remain MIT
A repeated day pattern that is not adjacent is written out again	the encoder collapses maximal runs of consecutive identical days only
A change of project repeats the activity letter, so <code>sBBBsCCCsD</code> stands	a run covers consecutive blocks sharing activity and project, and two template names need renaming
<code>:risky:</code> replaces <code>:speculative:</code> as the tag in a table	the stored class keeps the name <code>speculative</code> , so the schema, the views, both dashboards, the fixtures, and the elisp twin are untouched
<code>:speculative:</code> is no longer read	an old table is flagged as carrying an unread tag rather than misread, and the shipped tables were rewritten
<code>name.encode</code> joins <code>name.decode</code>	the editor can name the file it saves, and the command line gains the inverse function
A <code>dashboard</code> subcommand joins the command line	the two front ends offer the same actions
The table round-trip test comes before the editor writer	the one unrecoverable failure is caught first
<code>pytest-qt</code> joins the <code>dev</code> extra, and the Makefile exports <code>QT_QPA_PLATFORM=offscreen</code>	the suite runs without a display
A per-command database path stays	each command form carries its own <code>--db</code> widget
The Sphinx site gains a page for the interface	the graphical layer is documented where the rest of the package is

Table 2: The subcommands of the two packages and the options each accepts.

Package	Subcommand	Options
writing-habit	initdb	--db
writing-habit	plan import PATH	--week, --db
writing-habit	track import PATH	--format {csv,ics}, --db
writing-habit	track add	--day, --project, --minutes, --category, --start, --end, --note, --db
writing-habit	compare	--week, --plot, --db
writing-habit	dashboard	--week, --out, --db
writing-habit	history	--from, --to, --plot, --db
writing-habit	context set	--week, --tag, --note, --db
writing-habit	context clear	--week, --tag, --db
writing-habit	context list	--week, --db
writing-habit	seasons	--out, --db
writing-habit	name CODE	--table
writing-schedule	generate TABLE	--week, --day, --dir, --no-ics, --no-todo, --tz, --strict
writing-schedule	export TABLE	--week, --day, --dir, --no-todo, --tz, --strict
writing-schedule	sheets TABLE	--week, --day, --dir, --engine {reportlab,latex}, --format {pdf,org,both}, --per-day, --strict
writing-schedule	check TABLE	none
writing-schedule	template N	--out
writing-schedule	weeks	--dir

also carries a **check** subcommand that the status notes do not mention, and it is the natural backing call for the live overlap indicator in the editor.

3.2 The template table

The template is a single org table. Its first row names the days, its section rows name the activity class, its time-block rows carry one project code per day cell, and its legend rows map each code to a description and an optional risk class. The parser in `writing_schedule/parser.py` tests four row kinds in a fixed order and takes the first match, so the editor model must preserve that row order when it writes the file back.

Table 3: The four row kinds the parser recognizes, and how the editor represents each.

Row kind	Recognized by	Editor representation
Header	any cell after the first naming a day, such as M or Tue	column headers of the grid, fixed
Section	first cell of letters and spaces, optional trailing colon	a spanning group row, not editable as a block
Time block	a time range such as 04:00-05:30 in the first cell	an editable row of one combo box per day
Legend	first cell matching a short uppercase code then a colon	a row in the separate legend table

The risk class rides in the legend description as either (**safe**) or **:safe:**, and `writing_habit/name.py` strips it when reading the legend. The editor therefore edits the risk class as a combo box and writes the tag back in the form the file already used, so a round trip does not rewrite unrelated bytes.

3.3 What the tracker adds

The tracker reads the same table through `plan_import.import_org`, stores the schedule code taken from the file name, and keeps every planned block and every tracked session in one SQLite database. `name.decode` turns a code such as **4gAAeAsA-gWW** into a week of blocks, `name.summary` counts them, and `name.check_against_legend` reports whether each project letter resolves to a legend entry. Those three functions give the editor a live readout of what the current grid means, at no cost beyond calling them.

4 Architecture

4.1 In-process library calls, with the shell command on display

Two ways of driving the tools are possible. The interface can shell out to the `writing-habit` and `writing-schedule` executables, or it can import the packages and call their functions. The plan calls the library functions in a worker thread, for four reasons. The functions return Python objects, so the grid, the overlap list, and the report tables need no output parsing. Errors arrive as exceptions with tracebacks rather than as exit codes. Startup avoids a subprocess for every keystroke-driven revalidation. Testing needs no executable on the path.

The cost of that choice is that the graphical interface can drift away from the command line. Build step 2 removed the cost rather than managing it. The worker calls `writing_habit.cli.main(argv)` with the argument list the form built, so the preview above the run button is not merely equivalent to what runs; it is what runs. Drift is impossible while that holds, and a test asserts the stronger property, namely that the generated argument list parses back through the parser to the values the form holds. Every run records the command string in a session log pane the user can copy.

The interface drives two programs, so the runner keeps a table naming the module that holds each `main` function and imports it on demand. Resolving the program by name rather than by an import at load time is what lets the tracker run when the optional scheduler is absent. A tab whose commands belong to a missing program shows

the install line rather than vanishing, and the status bar says which program is missing, because a silently shorter window teaches the user nothing.

Two consequences of running in process deserve a note. The output capture replaces `sys.stdout` for the whole process, so a module lock allows one command at a time and the panel disables its run button while a command is in flight. The library uses `sys.exit` with a message to explain a missing optional dependency, so the runner turns a string payload into exit code one and writes the message to the captured output rather than discarding it. A user without `writing-schedule` installed therefore reads the install hint in the output pane instead of a traceback.

4.2 Module layout

Table 4: The proposed modules under `src/writing_habit/gui/` and the responsibility of each.

Module	Responsibility
<code>qt.py</code>	the single import site for Qt, so a later move to PyQt6 or PySide6 touches one file
<code>app.py</code>	the <code>main()</code> entry point, the <code>QApplication</code> , and the top-level exception hook
<code>mainwindow.py</code>	the window, the tab stack, the menu bar, the status bar, and the session log dock
<code>settings.py</code>	<code>QSettings</code> for the database path, the table directory, the output directory, and the recent files
<code>argspec.py</code>	introspection of an <code>argparse</code> parser into a list of command and option descriptors
<code>forms.py</code>	one option descriptor to one widget, plus the reverse read of the widget into an argument list
<code>runner.py</code>	a <code>QThread</code> worker that runs one command, captures <code>stdout</code> and <code>stderr</code> , and emits result or error
<code>command_panel.py</code>	the generated form for one subcommand, its command preview, its run button, and its output pane
<code>weekly_table.py</code>	the table as a document, classified by row kind, with the derived readings and a byte-exact writer. No Qt
<code>table_model.py</code>	a <code>QAbstractTableModel</code> over that document, mapping a grid cell back to its row and cell
<code>schedule_editor.py</code>	the grid, the four reading panels, and the open and reload controls
<code>legend_model.py</code>	a <code>QAbstractTableModel</code> over the legend rows, with code, description, and risk class
<code>report_view.py</code>	<code>ArtifactView</code> , which renders one file a command wrote, and <code>PreviewDock</code> , which keeps a tab per recent file

4.3 Tab layout

The window carries one tab per stage of the weekly loop, in the order the loop runs.

Table 5: The tabs, the widgets each holds, and the calls each makes.

Tab	Widgets	Backing calls
Schedule	the weekly grid and the legend table, above the generated <code>template</code> and <code>check</code> forms	<code>parse_text</code> , <code>find_overlaps</code> , <code>name.decode</code> , <code>name.encode</code> , and <code>writing-schedule template</code> and <code>check</code>
Generate	week or day picker, output directory, the <code>--no-ics</code> , <code>--no-todo</code> , <code>--tz</code> , and <code>--strict</code> controls	<code>writing-schedule generate</code> , <code>export</code> , and <code>weeks</code>
Sheets	engine and format combo boxes, the per-day check box, a page preview	<code>writing-schedule sheets</code>
Plan	table picker, week picker, database picker	<code>plan_import.import_org</code>
Track	a file importer and a hand-entry form for one session	<code>track.csv_actuals.import_csv</code> , <code>track.ics_actuals.import_ics</code> , <code>track.manual.add_session</code>
Compare	week picker, the text report, the optional plot, the dashboard preview	<code>compare.report.render_week</code> , <code>report.write_plot</code> , <code>dashboard.write_dashboard</code>
History	a from and to range, the text series, the plot	<code>compare.history.render_text</code> , <code>history.write_plots</code>
Context	a week picker, a tag combo box, a note field, the tag list	<code>context.set_tag</code> , <code>clear_tag</code> , <code>list_tags</code>
Seasons	an output path and a preview	<code>seasons.write_seasons</code>

5 The schedule editor

5.1 The grid

The grid is a `QTableView` over `table_model.py`, which reads `weekly_table.py`. Columns are the days found in the header row, so a six-day table and a seven-day table both work without a setting. Rows follow the file order, which keeps section rows in place. A section row renders bold and shaded and refuses edits.

The document underneath is the part that matters. It keeps every line of the file, classified by the four row kinds in the order the scheduler parser tests them, because that parser takes the first match and a different order here would give the editor a different table from the one the tools read. Two properties are asserted over every shipped table rather than assumed. The document sees the same blocks the scheduler sees, and an untouched document writes back byte for byte. The second property is the round-trip test the build order puts before the writer, and it holds for all thirteen tables in `templates/` and `examples/`.

A time-block cell is edited through a `QStyledItemDelegate` presenting a combo box of the legend codes plus a blank entry, so a cell rarely holds a code the legend does not define, though the box stays editable for a writer who wants to type a code and add its legend row after. A section row and the time column refuse an edit, because they belong to the file rather than to the week.

The legend has its own table below the grid, with the code, the description, and the risk tag as a choice of **none**, **safe**, and **risky**. The choice shows the tag a writer types rather than the class the database stores, because the table is the thing being edited. Writing **risky** puts `:risky:` in the file and yields the class the database calls **speculative**.

Every edit rewrites exactly one line. A value that fits its column keeps the table aligned, and a longer value widens its own slot alone, which leaves the table misaligned until the writer realigns it in Emacs. Reflowing lines they did not touch would be worse, because the diff would then hide the change they made.

Three edits change the shape of the week rather than one cell. Adding a time block inserts a row under the current section and asks for the time range with a validated line edit. Adding a section appends a section row and one block row. Adding a day is out of scope for the first release, because the day set comes from the header row and the parser reads it once.

5.2 The live readouts

One revalidation pass runs when a table opens, and step 5 will run the same pass after every edit, which is cheap because a week holds a few dozen blocks. The pass fills four panels beside the grid.

The name panel gives the canonical file name for the grid, the week the code decodes to, and the block counts by activity and by project. When the file name on disk differs from the canonical form, the panel says so, which is how a writer notices that a table has drifted from its name.

The clashes panel lists the pairs from `writing_schedule.overlap.find_overlaps`, the tab carries the count, and the offending cells take a warning tint. This is the graphical equivalent of the `check` subcommand and of the `--strict` guard.

The totals panel shows planned minutes by day, by project, and by activity, with the week total. It also names any section header that maps to no activity category, because the plan importer silently counts such a section as generative, and a writer should know that before the numbers reach the database.

The legend panel shows the output of `name.check_against_legend`, so an unknown or ambiguous project letter is visible before the file is used. It also lists legend entries the week never uses, the codes defined more than once, and the risk tags that name no class.

One reading has no answer for some tables, and saying so plainly is the useful behavior. A file name gives one letter to one block, so a legend code of more than one letter, such as **EM** for email, cannot appear in a cell of a named table. The name panel then says which code is at fault and that the naming rules answer it with a single-letter alias, because only the writer knows which letter is free.

5.3 What the real week showed

`examples/aug24.org` is a real weekly table rather than a demonstration, so running the document over it was the first honest test. The table offers a menu of one hundred and twenty-six candidate time blocks and fills five of them, all on the Thursday, which the readings handle correctly. The week totals five hundred and seventy planned minutes, nothing clashes because the blocks merely touch, and the canonical name is `3o-gHgAeCsIsB`, where the three open days are Monday through Wednesday. An empty cell is not a plan, and only a lettered cell becomes a block.

Three faults in the data surfaced that no synthetic example would have shown.

Table 6: What the real table revealed, and what each fault costs.

Fault	Cost	Response
The legend defines H twice, for 1006A1rx0pt and 2174ComputationalCrystallography, and skips 0	one project is silently unreachable, and the Thursday H block resolves to the first	the legend panel names the code and both descriptions; the writer chooses
G and P carried <code>:spec:</code> , which names no risk class	both projects fell out of the barbell comparison, which reads only the tags it knows	the legend panel reports a tag that was not read; the table now reads <code>:risky:</code>
<code>name.read_legend</code> kept the last definition of a duplicated code while the plan importer keeps the first	one table gave two different descriptions for one project, depending on the command	<code>read_legend</code> now keeps the first, matching the importer, with a test asserting the agreement

The third fault was in the code rather than the data, and it existed before this work. The scheduler resolves a legend code in the manner of elisp `assoc`, which returns the first matching entry, so the importer and the reader disagreed on exactly the tables where it matters.

5.4 The rounding that made a page depend on its machine

`tests/test_seasons.py` failed from the first day of this work, and the cause turned out to be worth more than the test. The grouping views ended with `ROUND(1.0*SUM(actual_min)/SUM(planned_min), 2)`, and both dashboards printed that column. One row of the fixture is 510 minutes against 1200, which is exactly 0.425, and SQLite does not agree with itself across versions on such a value. Version 3.37 gives 0.43 and version 3.45 gives 0.42, so one database rendered two different pages on two machines, and the golden that both ports assert against failed on whichever machine had not written it.

The fix is to compute every displayed ratio where the page is built rather than in the query. Python formats the same double the same way everywhere, and so does the elisp twin, so the page a database yields no longer depends on the SQLite build underneath. The views keep their `adherence` column, because nothing forces a stored view to change and the fixture databases carry the old definition anyway.

The committed goldens did not move, which is the useful part. The computed value reproduces both pages byte for byte, so the fix removed a dependency rather than redefining the numbers. Two tests hold the line: one asserts that a view column which disagrees never reaches the page, and one pins the ratio of the value the two SQLite builds disputed.

5.5 The encoder

`name.py` decoded a code into a week and had no inverse, so the editor had no way to name the file it saves. `name.encode(week)` and `name.encode_day(blocks)` now supply that inverse. They live in the library rather than in the graphical layer, because the command line and the Emacs Lisp twin want the same function.

The canonical spelling follows `docs/table-file-naming-rules.org`. A maximal run of identical consecutive days collapses into one group carrying the day count, a count of one is omitted, trailing open days are dropped because they are implied, and an interior open day is written as `o`. Two days that share a pattern without being adjacent are written out again, which is the rule you chose, so a Monday, Wednesday, Friday week reads `gA-o-gA-o-gA`. The property `encode(decode(code)) = code` holds for every canonical code, and the corpus test asserts the weaker semantic property `decode(encode(decode(name))) = decode(name)` for every file name in `templates/`.

A second rule was needed for a run of blocks that share an activity while the project changes. Both spellings decode to the same week, so the six canonical rules did not choose between them, and the corpus splits evenly. Two template names repeat the activity letter at each change of project, giving `sBBBsCCCSD`, and two merge the run, giving `gAAB`. The remaining eight names never change project inside a run, so they are silent on the question.

The repeated activity letter is the canonical form, which is your decision. A run therefore covers consecutive blocks that share both the activity and the project, so three support blocks on B followed by three on C followed by one on D read `sBBBsCCCSD` rather than `sBBBCCCD`. The rule keeps every project visible in the file name, which is the point of the name. The two names that use the merged spelling inside a generative run become non-canonical, so `gAABeAsA-gAABeBsB-gBBaEAsB-gBBaEBSW-gWsA.org` and its six-day sibling would be renamed to `gAAgBeAsA-gAAgBeBsB-gBBgAeAsB-gBBgAeBSW-gWsA.org`. Nothing renames them automatically, because the tracker groups weeks by the stored code.

5.6 Saving

Saving writes the document back to the file it came from, and **Save as** offers the canonical name `<code>.org` when the grid has one. A file that was not edited saves byte for byte, and a file that was edited differs only on the lines that changed. Those two properties were tested before the writer existed, because they are the failure a writer cannot recover from.

New from template asks for a project count and calls `writing_schedule.template.template_string`, so a blank week comes from the scheduler's own scaffold rather than from a second copy of the layout here.

Rename to canonical moves the file to the name its grid gives it. The button offers itself only when the file is saved and misnamed, and the rename refuses to overwrite a different file, refuses a week that has no canonical name, and refuses while edits are unsaved. The action matters more than it looks, because the tracker groups weeks by the schedule code stored at plan import, and that code comes from the file name, so a drifted name files the week under the wrong plan shape and the seasonal comparison then groups it with the wrong weeks.

6 The command panel

6.1 Generating forms from argparse

`argspec.py` walks a parser and yields a descriptor for every subcommand and every option. The walk reads `parser._subparsers`, then for each subparser reads `_actions`, skipping the help action. Each action gives the option strings, the **dest**, the **help** text, the **choices**, the **type**, the **required** flag, and whether it is a **store_true** action. Private attributes are involved, so the walk is confined to `argspec.py` and is covered by a test that fails loudly when a future Python changes the layout.

The sets are small tables in `forms.py`, and they are keyed by the primary flag rather than by **dest**, because two commands can share a **dest** while meaning different things. `history --from` and `track add --start` both carry the **dest start**, yet one is a calendar date and the other is a clock time. A test asserts that pair, because it is the case a **dest** table would silently get wrong.

One more table holds suggestions. `--category` and `--tag` take a value the parser does not restrict, while the schema and the documented vocabulary do have expected values, so those two get an editable combination box that offers **generative**, **editing**, and **support** for the first and the event vocabulary for the second. A new option that falls outside every table still produces a working line edit, so an unrecognized flag degrades to a usable widget rather than to a crash.

Table 7: The rule that maps an argparse action to a widget.

Action shape	Widget	Example
<code>action"store_true"=</code>	<code>QCheckBox</code>	<code>--strict, --no-ics, --per-day</code>
<code>choices present</code>	<code>QComboBox</code>	<code>--format, --engine</code>
<code>type=int</code>	<code>QSpinBox</code>	<code>--minutes</code>
<code>dest in the date set</code>	<code>QDateEdit</code> with a calendar popup and a "today" button	<code>--week, --day, --from, --to</code>
<code>dest naming a directory</code>	line edit plus a directory chooser	<code>--dir</code>
<code>dest naming a file or a positional path</code>	line edit plus a file chooser with a suffix filter	<code>--out, --plot, --table, path</code>
<code>anything else</code>	<code>QLineEdit</code>	<code>--tag, --note, --tz</code>

An optional value needs a way to say nothing at all. An empty line edit omits its option, and a date, a time, or a count carries a small check box that decides whether the option is sent, because those widgets always show something.

One option needs the opposite. `writing-schedule --tz ""` asks for floating calendar times, so an empty value there is a request rather than an omission. That flag gets a line edit with the same send box, and the argument builder treats “None” as the omission and an empty string as a value.

6.2 Running a command

The run button collects the widget values into an argument list, renders the preview string, and hands a callable to `runner.py`. The worker runs on a `QThread` and communicates only through signals, because touching a widget from a worker thread is the standard way to crash a Qt program. The worker redirects `sys.stdout` and `sys.stderr` with `contextlib.redirect_stdout` so that the printed output of the existing functions reaches the log pane unchanged.

The database needs care. `sqlite3` connections belong to the thread that created them, so the worker calls `db.connect` itself and closes the connection when the task ends. The main thread keeps no long-lived connection. Read-only panels that need a quick query open a short connection on the main thread and close it immediately, which is acceptable at this data volume.

6.3 Preferences and defaults

The database path, the template directory, the output directory, the time zone, and the last used week persist through `QSettings`. Every form seeds its widgets from those settings, so the common case of running `plan import`, then `compare`, then `dashboard` for the same week needs no retyping.

The `--db` widget stays on every command form, which is your decision and which matches the command line, where `--db` is required on every subcommand that touches the database. Two habits keep a per-command path from becoming a source of silent mistakes. The remembered path seeds every form, so the widgets agree unless you deliberately change one. The status bar shows the path of the database the last command actually used, together with its row counts, so a command that ran against the wrong file is visible immediately rather than three commands later.

7 Seeing what a command wrote

A command that writes a file used to end with a line of text naming a path, and the writer then went looking for it. The preview dock removes that step. It holds one tab per recent file, newest last, and it appears the first time a command produces something rather than occupying the window before then.

The panel knows what its run produced in two ways, and it needs both. The paths given as `--out` and `--plot` come from the form, so nothing has to be parsed for the tracker commands. The scheduler instead takes a directory

and derives the file names, so those paths are read from the `Wrote` lines the command prints, taking the last token of each line and keeping it only when it names a file on disk. A token that is not a file is dropped, so a sentence that merely begins with the word is harmless.

Table 8: What each kind of output does in the preview.

Output	Shown as	Note
The weekly dashboard and the seasons page	the web engine when it is installed, and the rich-text widget otherwise	the fallback says what it cannot render
The compare and history plots	the image, with its pixel size	scrolls when it is larger than the dock
The schedule <code>.org</code> , the <code>.ics</code> , the sheet <code>.org</code> , and the <code>.tex</code>	plain text	a file over two hundred kilobytes is named rather than loaded
The sheet PDFs	the name and nothing else	Qt has no PDF viewer here, so the desktop opens it

Every kind offers `Open outside`, which hands the file to the program the desktop would use. That button is the honest answer for the dashboards, because they use CSS that the rich-text widget of Qt renders only roughly.

7.1 The rule the web engine imposes

`PyQtWebEngine` ships separately from `PyQt5`, so most installations lack it and the fallback is the normal path. Where it is installed, Qt imposes an order that is easy to get wrong and quiet when it is. The web-engine module must be imported, and the shared OpenGL context attribute must be set, before the `QApplication` exists. Import it later and Qt prints a warning and refuses, which leaves a machine that has the faithful renderer using the rough one forever, with nothing in the interface to say why.

The startup therefore calls one preparation function before building the application object, and the probe caches its answer so a later call still gives the right one. A test asserts the order inside `main`, because the failure is invisible at run time.

8 Packaging

8.1 The optional extra

The graphical layer ships inside `writing-habit` behind an extra, so the wheel and the command line stay free of a Qt dependency.

```
[project.optional-dependencies]
gui = ["PyQt5>=5.15", "writing-schedule>=0.3.1"]
preview = ["PyQtWebEngine>=5.15"]
```

```
[project.scripts]
writing-habit = "writing_habit.cli:main"
writing-habit-gui = "writing_habit.gui.app:main"
```

The web engine sits in its own extra rather than in `gui`, because `Open outside` already answers the same need and the engine is a large dependency for a convenience. It is also GPL, so an installation that wants neither GPL package simply installs neither extra.

Three rules keep the extra honest. No module outside `writing_habit/gui/` may import Qt. The `gui` package imports Qt only through `gui/qt.py`. The `writing-habit-gui` entry point catches the import error and prints the installation hint `pip install "writing-habit[gui]"` rather than a traceback.

The `gui` extra depends on `writing-schedule` because the editor uses its parser, its overlap detector, and its template scaffold. That package is already an optional dependency under the `plan` extra, so the two extras should be documented together.

8.2 The choice of PyQt5 and the split license

PyQt5 is the chosen binding, and it is available to the interpreter you will use. PyQt5 is in maintenance while PyQt6 and PySide6 receive the new work, so the single import site in `gui/qt.py` earns its keep as insurance. A later move changes the import names and a small number of enumeration references, which is a one-file change when nothing else imports Qt directly.

PyQt5 is distributed under the GPL, so the package now carries two licenses. The `pyproject.toml` comment above the extra records that, and three places need the same sentence. The README needs a license section that states that the library and the command line are MIT while the optional graphical extra pulls in PyQt5 under the GPL. The About dialog of the interface needs the same statement. The Sphinx installation page needs it beside the `pip install "writing-habit[gui]"` line. A user who installs the package without the extra is unaffected, which is the reason the extra exists.

9 Testing

The test suite runs headless with `QT_QPA_PLATFORM=offscreen`, and `pytest-qt` joins the `dev` extra. Six groups of tests carry the weight.

Table 9: The test groups, what each protects, and the failure each prevents.

Group	What it asserts	Failure it prevents
Round trip	every file in <code>templates/</code> and <code>examples/</code> loads into the model and writes back byte-identical	silent rewriting of a user's table
Model editing	a cell edit, a row insertion, and a legend edit produce the expected table text	corruption of row order
Encoder	<code>encode(decode(code)) = code</code> for canonical codes, and the semantic round trip for every template name	a wrong file name, which becomes a wrong grouping key
Argument specification	every subcommand and option in both parsers produces a descriptor and a widget	a new flag that the interface cannot reach
Command preview	the rendered string parses back through the parser to the same namespace	a preview that lies about what will run
Worker	a failing task emits the error signal and leaves the window responsive	a frozen window on a bad path

The round-trip group is worth writing first, because it also documents the table format in executable form.

10 Build order

Each step below leaves a working program, so the work can stop at any step and still be useful. Every step is complete.

1. [DONE] The library work the graphical layer depends on, none of which imports Qt. `name.encode` and `name.encode_day` with their tests, the `dashboard` subcommand with its test, the `gui` extra and `pytest-qt` in the `dev` extra, and a repository Makefile whose test target exports `QT_QPA_PLATFORM=offscreen`.
2. [DONE] Add `gui/qt.py`, `gui/settings.py`, `gui/app.py`, and `gui/mainwindow.py` with a tab per stage of the loop, the session-log dock, the `About` dialog carrying the split license, and the `writing-habit-gui` entry point. Two test modules come with it, one exercising the window offscreen and one asserting that Qt never leaves the `gui` package. This proves the packaging and the import guard before any feature exists.
3. [DONE] Build `argspec.py` and `forms.py` against `writing_habit.cli.build_parser()`, and render the Plan, Track, Compare, History, Context, and Seasons tabs from the descriptors. Add `runner.py` and wire the panels to the log pane. All twelve tracker subcommands now run by point and click, and the status bar names the database the last run used.
4. [DONE] Extend the same machinery to `writing_schedule.cli.build_parser()` for the Schedule, Generate, and Sheets tabs. The runner gained a table of the two programs and resolves the `main` function by name, so the tracker keeps working when the scheduler is absent and those tabs then explain how to install it. All six scheduler subcommands run by point and click.
5. [DONE] Build the read-only schedule grid, with the name, clashes, totals, and legend panels. The document in `weekly_table.py` carries the round-trip property the writer of step 5 needs, and `name.parse_legend_cell` now serves both the file reader and the in-memory grid, so one rule reads a legend row. This step shows a table and writes nothing.
6. [DONE] Make the grid editable with the combo-box delegate, add the legend editor with its risk-tag choice, and add the writer that rewrites one line per edit. Add `Save`, `Save as` with the canonical name, and `New from template`, which calls `template_string`.
7. [DONE] Wire the canonical file name into the interface and into the twin. `name.encode` and its property test landed in step 0, and `Save as` offered the canonical name in step 5, so this step closed the two gaps that left. The editor gained a rename action, because a name that has drifted files the week under the wrong shape and nothing acted on the panel's warning. The Emacs Lisp twin gained `writing-habit-name-encode` and `writing-habit-name-encode-day`, so both ports can name a week, and a cross-check confirms they name every week alike.
8. [DONE] Add the preview dock for the dashboards, the plots, and the text files, with `Open outside` for everything and the web engine where it is installed.
9. [DONE] Add `docs/gui.md` to the Sphinx site, place it in the `index.md` table of contents beside `cli.md`, and illustrate it with two screenshots taken from the running interface. State the split license there, on the installation page, and in the README. Add the `preview` extra for `PyQtWebEngine`, which stays optional even with `gui`.

11 Risks and open questions

12 Follow-up checklist

- [X] Confirm that PyQt5 is available to the Python interpreter you will use.
- [X] Ship the graphical layer inside `writing-habit` behind the `[gui]` extra.
- [X] Choose PyQt5 under the GPL, and record the split license in `pyproject.toml`.
- [X] Resolve the canonical-code rule: a non-adjacent repeated day is written out again.
- [X] Add `name.encode` and `name.encode_day` with their round-trip tests.
- [X] Add the `dashboard` subcommand to `writing_habit/cli.py` for parity.
- [X] Add `pytest-qt` to the `dev` extra.
- [X] Add a Makefile whose test target exports `QT_QPA_PLATFORM=offscreen`.
- [X] Keep a per-command database path on every command form.

Table 10: The risks that deserve a decision before coding, and the response to each.

Risk	Why it matters	Response
Private <code>argparse</code> attributes	<code>_actions</code> and <code>_subparsers</code> are not public API	confine the access to <code>argspec.py</code> and test it explicitly
The repeated activity letter	<code>sBBBsCCCsD</code> and <code>sBBBCCCD</code> decode alike, and the six canonical rules did not choose	the repeated letter is canonical; rename the two merged template names
Byte-exact round trip	column padding and trailing spaces vary between hand-edited files	normalize on load, and warn the user when the first save will reformat
GPL of PyQt5 beside MIT	a split license needs a plain statement	state it in the README, the About dialog, and the installation page
Threading with <code>sqlite3</code>	a connection shared across threads raises at runtime	open and close the connection inside the worker
A per-command database path	one form can point at a different file than the last	seed every form from the remembered path, and name the database of the last run in the status bar
Feature drift from the command line	a graphical action with no command-line equivalent splits the two ports	require a subcommand for every action, which step 0 restored for the dashboard
The Emacs Lisp twin	the elisp package has no matching interface	treat the graphical layer as Python-only, and keep every shared rule in the library; the encoder and the <code>:risky:</code> tag are ported, and the two ports name every week alike
A legend code defined twice	the two readers disagreed, and one project becomes unreachable	<code>read_legend</code> now keeps the first, and the legend panel reports the duplicate
An abbreviated risk tag	<code>:spec:</code> names no class, so the project leaves the barbell comparison without a word	the legend panel reports a tag that was not read
The web-engine import order	importing it after the <code>QApplication</code> exists leaves the faithful renderer unused and says nothing	one preparation call at startup, a cached probe, and a test on the order inside <code>main</code>
Rounding in the golden fixtures	<code>ROUND</code> in SQL rounded 0.425 differently between SQLite builds, so one database gave two pages	fixed: every displayed ratio is computed in Python, in both ports, and the goldens stand unchanged

- [X] Fix the canonical run rule: a change of project repeats the activity letter.
- [X] Build the window shell, the packaging, and the two graphical test modules.
- [X] Generate the command forms from argparse, and run every tracker subcommand.
- [X] Extend the same machinery to writing-schedule for the Generate and Sheets tabs.
- [X] Build the weekly-table grid in the Schedule tab, above the template and check forms.
- [X] Write the table round-trip test, which passes for all thirteen shipped tables.
- [X] Give EM and TT single-letter aliases so examples/my-week.org can be named.
- [X] Decide which project keeps the code H in examples/aug24.org, and give the other one O.
- [X] Replace :spec: in aug24.org, which now reads :risky:.
- [X] Read :risky: as the tag for the speculative class, and stop reading :speculative:.
- [X] Rewrite the tag in the nine shipped tables, the README, and tracking-formats.md.
- [X] Make the grid and the legend editable, and add the writer, Save, Save as, and the scaffold.
- [X] Port the :risky: tag to writing-habit-el, so the two ports read a table alike.
- [X] Port name.encode to writing-habit-el, with ERT tests and a cross-port check.
- [X] Add a rename action, so the canonical file name is actionable rather than advisory.
- [X] Add the preview dock for the dashboards, the plots, and the text files.
- [X] Keep PyQtWebEngine in its own preview extra, optional even with gui.
- [X] Add docs/gui.md, place it in the toctree, and illustrate it with two screenshots.
- [X] State the split license in the README and on the Sphinx installation page.
- [X] Fix tests/test_seasons.py, whose golden depended on the SQLite build underneath.
- [] Review the diff in both repositories, then commit; the elisp twin changed too.
- [] Regenerate the two GUI screenshots under assets/images with the CC BY metadata, if they should 1.
- [] Consider a writing-habit-gui section in the elisp twin's README, saying the interface is Python-
- [] Rename the two template files that merge a project change inside a generative run.
- [] Port name.encode to the Emacs Lisp twin so the two ports name files alike.